

Universidad Nacional Autónoma de México



Facultad de Ingeniería

División de Ingeniería Eléctrica



Laboratorio Computación Gráfica e Interacción
Humano-Computadora (6590)

Práctica 9: Animación

Nombre: Calderón Gutiérrez Victor Emiliano

No. Cuenta: 423097331

Grupo (laboratorio): 03

Grupo (teoría): 02

Semestre 2026-1

Fecha de entrega: 02/11/2025

Calificación: _____

1.Ejercicios

1.1. Ejercicio 1

El primer ejercicio para esta práctica trataba de usar el modelo de la puerta y el arco que se tenía que separar, para así tener las dos puertas, el arco y el letrero por separado, esto fue la parte más sencilla, ya que se hizo como en prácticas anteriores al separar modelos y exportarlos por separados.

De esta forma primero se deben crear las variables para los modelos.

```
Model Dragonalaizq_M;  
Model Tiamat_M;  
Model Arco_M;  
Model Puerta_izq_M;  
Model Puerta_der_M;  
Model Letrero_M;  
Skybox skybox;
```

Una vez ya se tienen las variables entonces se le cargan los modelos procesados en Blender.

```
327 Dragonalaizq_M.LoadModel("Models/alaizquierda.obj");  
328 Arco_M = Model();  
329 Arco_M.LoadModel("Models/arco.obj");  
330 Puerta_izq_M = Model();  
331 Puerta_izq_M.LoadModel("Models/puerta_izq.obj");  
332 Puerta_der_M = Model();  
333 Puerta_der_M.LoadModel("Models/puerta_der.obj");  
334 Letrero_M = Model();  
335 Letrero_M.LoadModel("Models/letreto.obj");  
336
```

Por último, se renderizan tal y como lo hemos hecho anteriormente.

```

// Ejercicio 1: Separar del arco la parte del letrero y las puertas o rejas
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 20.0f));
arco = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
Arco_M.RenderModel();

//puerta izquierda
model = arco;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.5f));
model = glm::rotate(model, glm::radians(-rotPuerta), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Puerta_izq_M.RenderModel();

//puerta derecha
model = arco;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.75f + movPuerta)); // se desliza hacia la derecha
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Puerta_der_M.RenderModel();

//letrero
model = arco;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Letrero_M.RenderModel();

```

1.2. Ejercicio 2

Para el ejercicio 2 se pidió algo un tanto sencillo, ya que se basaba mucho en lo que se hizo en el ejercicio de clase. Se pedía hacer que un texto se moviera como si fuera un panel LED/LCD.

Primero, para lograr eso se hizo uso de la tipografía y además se creó un archivo donde venían las palabras que se pedían.



Ya que se tenía eso, entonces se reutilizó el código que se tenía como base del ejercicio de clase.

```

/* Ejercicio 2: Hacer que en el arco que crearon se muestre la palabra :
PROYECTO CGEIHIC desplazándose las letras de derecha a izquierda como
si fuera letrero LCD/LED de forma cíclica*/

toffsetletrou += 0.0005;
if (toffsetletrou > 1.0)
    toffsetletrou = 0.0;
toffsetletrou = 0.0;
toffset = glm::vec2(toffsetletrou, toffsetletrou);
model = arco;
model = glm::translate(model, glm::vec3(-0.5f, 6.5f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
LetreroTexture.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Pero si nos damos cuenta, este código se modificó creando nuevas variables llamadas `toffsetletrou` y `toffsetletrou`, las cuales nos ayudaron a mover el texto. Posteriormente, se utilizó la misma lógica que lo que se nos dio en un inicio en el ejercicio de clase, solo que ahora se hicieron incrementos más pequeños para que esto se notara correcto.

1.3. Ejercicio 3

Por último, este ejercicio si resultó más complicado, se pedía que con un botón se lograra abrir una puerta, pero implementando una puerta que se desliza y otra que rota. Para esto primero se creó el método para hacer esto en `Window.cpp/h`.

```

}

if (key == GLFW_KEY_A && action == GLFW_PRESS) {
    theWindow->abrirPuertas = !theWindow->abrirPuertas;
}

GLboolean getabrirPuertas() { return abrirPuertas; }
~Window();

private:
    GLboolean abrirPuertas;

```

Ya que se tenía esto, usé la misma lógica que tenía en el ejemplo, un código que define cuando la puerta se abre y se cierra.

```

/*Ejercicio 3: Hacer que al presionar una tecla una de las puertas/rejas
se abra por medio de una rotación y la otra puerta se abra por medio de un deslizamiento.*/

if (mainWindow.getAbrirPuertas()) {
    if (rotPuerta < 90.0f) rotPuerta += velPuerta * deltaTime;
    if (movPuerta < 3.0f) movPuerta += 1.0f * deltaTime;
}
else {
    if (rotPuerta > 0.0f) rotPuerta -= velPuerta * deltaTime;
    if (movPuerta > 0.0f) movPuerta -= 1.0f * deltaTime;
}
}

```

Esto permite que al presionar la tecla A las puertas abran o cierren como se necesite.

2. Problemas

Creo que el problema más fuerte que se tuvo fue al momento de inicializar las variables para hacer que funcionara el abrir y cerrar de las puertas. Me basé en lo que se hizo en el ejemplo de clase, pero aun así cada que comienza mi código a correr las puertas aparecen al otro lado del mapa, como si estas estuvieran mal colocadas, pero al presionar la A una vez estas se acomodan y luego únicamente se mueven en el rango establecido. Esto me dio muchos dolores de cabeza por no encontrar solución.

3. Conclusión

Cada vez más cerca del proyecto final y cada vez se nota que ya estamos implementando todo lo que hemos hecho durante el semestre. El lograr hacer esto nos ayuda a entender como las animaciones funcionan, como es que estas se pueden implementar no solo por programas específicos para eso, sino que también se pueden hacer con código en C++. Por lo que esto servirá para optimizar y mejorar el código y el como es que se ven las animaciones. Gracias a esta práctica, entendí como trabajar con varias variables, como hacerlo con DeltaTime y como es que existen muchos métodos, muchas formas y se pueden hacer muchas cosas para lograr que las cosas se parezcan un poco a como se ven en nuestras cabezas.

4. Bibliografía

- Ing. Jose Roque RG. (2025e, octubre 27). Lab 3 práctica 9 Animación [Video].

YouTube. Recuperado 2 de noviembre de 2025, de

<https://www.youtube.com/watch?v=KUHzBz2bhc>